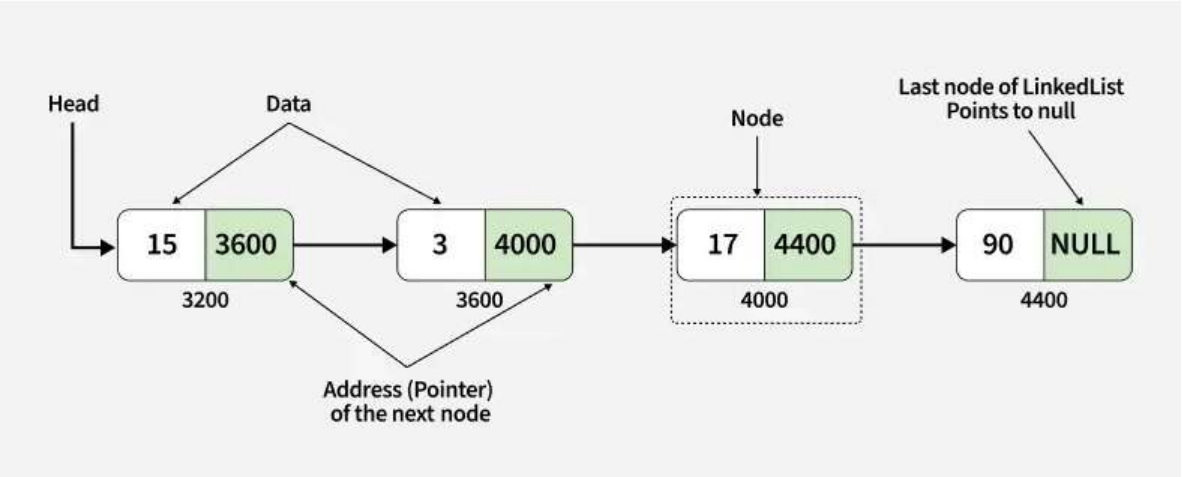
Linked List

Thursday, May 14, 2026 8:09 AM

A linked list is a type of linear data structure individual items are not necessarily at contiguous locations. The individual items re called nodes and connected with each other using links. A node contains two things, first is data and second is a link the connects it with another node. The first node is called the head node and we can traverse the whole list using this head and next links.



Linked List:

- Data Structure: Non-contiguous
- Memory Allocation: Typically allocated one by one to individual elements
- Insertion/Deletion: Efficient
- Access: Sequential

Array:

- Data Structure: Contiguous
- Memory Allocation: Typically allocated to the whole array
- Insertion/Deletion: Inefficient
- Access: Random

Singly linked list

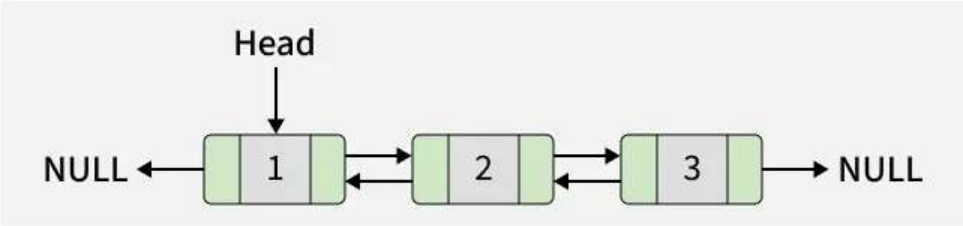
It consists of nodes where each node contains a data field and a reference to the next node. The next of teh last node is NULL.

```
# Definition of a Node in a singly linked list
class Node:
    def __init__(self, data):

        # Data part of the node
        self.data = data
        self.next = None

# constructor to initialize a new node with data
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None

# Create the first node (head of the list)
head = Node(10)
```

```
# Link the second node
head.next = Node(20)

# Link the third node
head.next.next = Node(30)

# Link the fourth node
head.next.next.next = Node(40)

# printing linked list
temp = head
while temp is not None:
    print(temp.data, end=" ")
    temp = temp.next
```

Doubly linked list

It offers several advantages. The main one is that allows for efficient traversal of the list in both directions. This is because each node of the list contains a pointer to teh previous node and a pointer to teh next node.



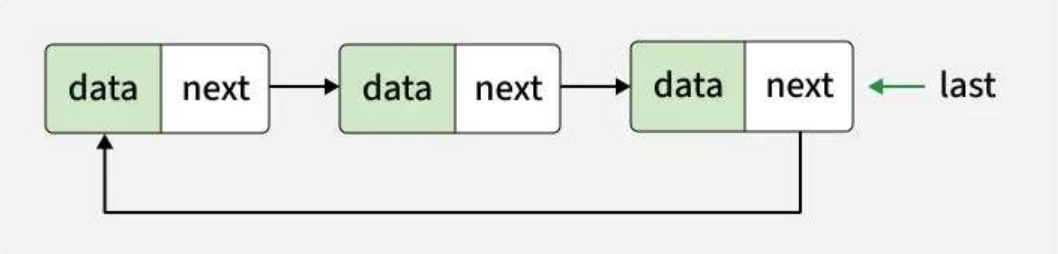

```
class Node:

    def __init__(self, data):
        # To store the value or data.
        self.data = data

        # Reference to the previous node
        self.prev = None

        # Reference to the next node
        self.next = None

class Node:
    def __init__(self, value):
        self.data = value
        self.prev = None
        self.next = None

if __name__ == "__main__":
    # Create the first node (head of the list)
    head = Node(10)

    # Create and link the second node
    head.next = Node(20)
    head.next.prev = head

    # Create and link the third node
    head.next.next = Node(30)
    head.next.next.prev = head.next

    # Create and link the fourth node
    head.next.next.next = Node(40)
    head.next.next.next.prev = head.next.next

    # Traverse the list forward and print elements
```

```
    temp = head
    while temp is not None:
        print(temp.data, end="")
        if temp.next is not None:
            print(" <-> ", end="")
        temp = temp.next
```

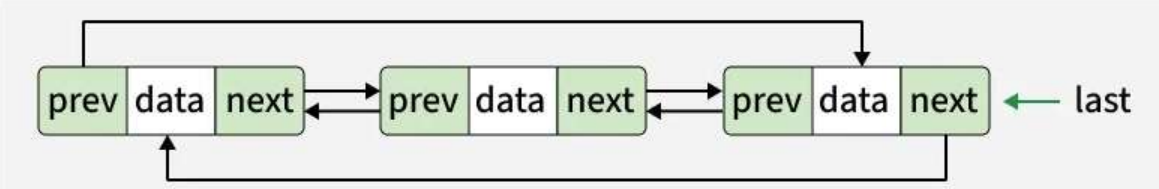
Circular linked list

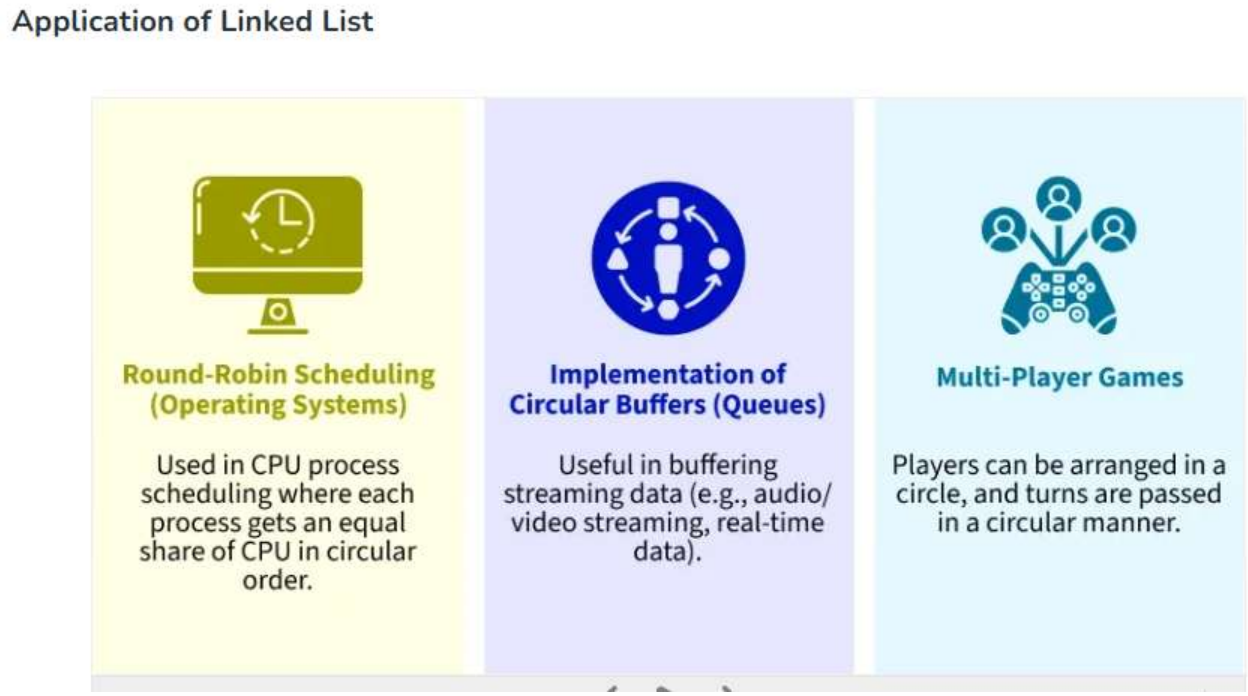
It is a structure where the last node points back to teh first node, forming a closed loop. We can create a circular linked list from both singly and soubly ones.

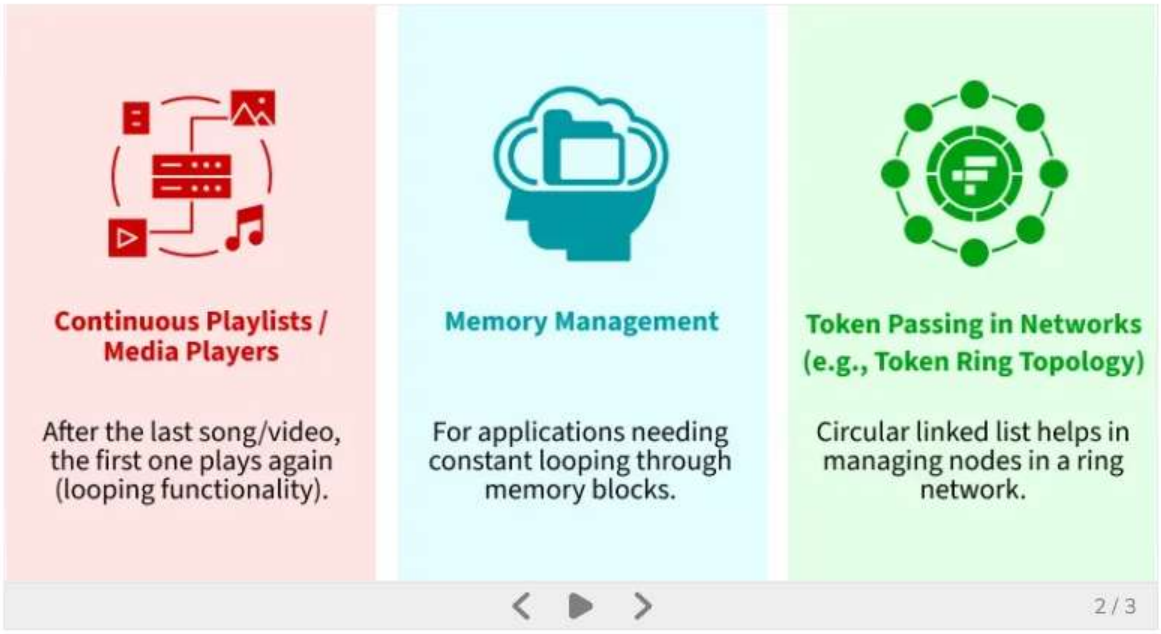
In the first case each node has just one pointer called next pointer, teh next pointer of teh last node points back to the first node.

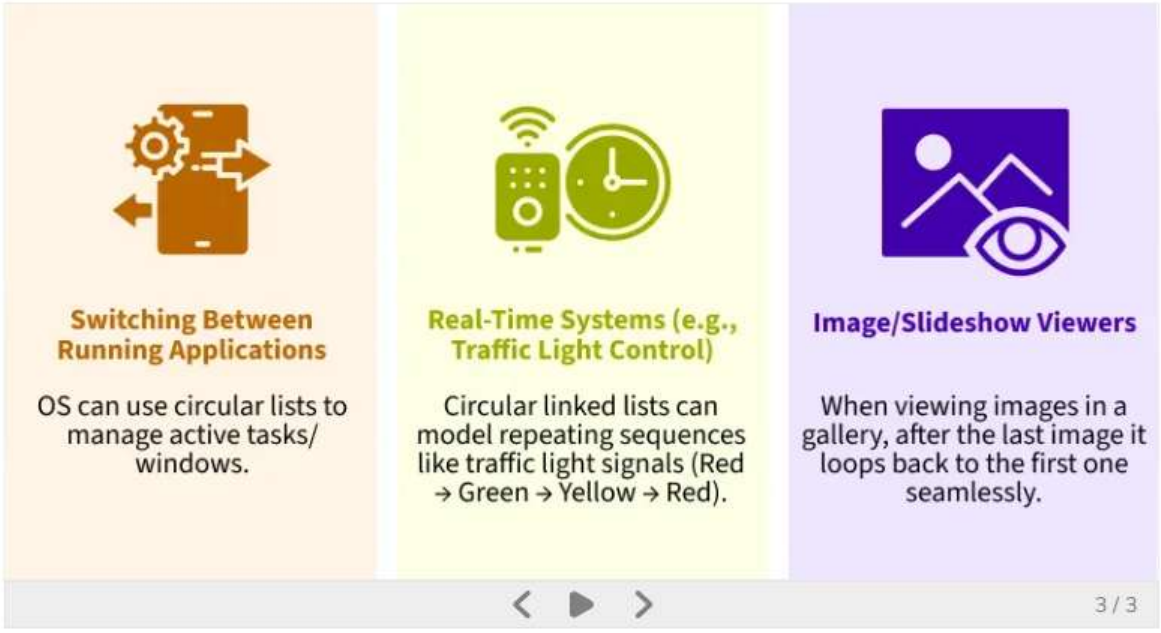


In the second case, eahc node has two pointers prev and next. Teh prev pointer points to teh previous node and teh next one to teh next node. Here, in addition to teh last node storing the address of teh first node, teh first node will also store the address of teh last node.









Operations

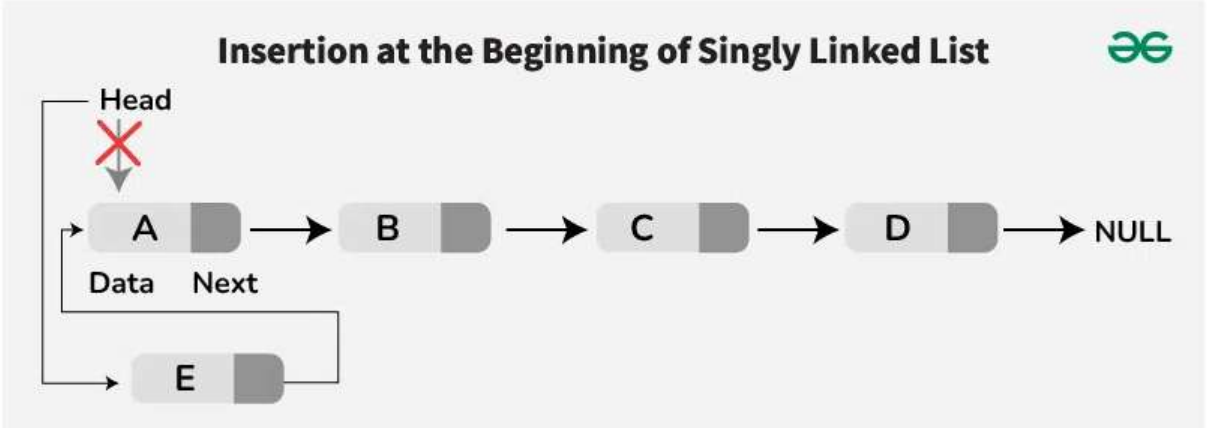
- Length
  - Iterative approach: time complexity O(1), space O(1)
  - Recursive: time complexity O(n), space O(n)

```
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None

# Recursively count number of nodes in linked list
def count_nodes(head):
    # Base Case
    if head is None:
        return 0

    # Count this node plus the rest of the list
    return 1 + count_nodes(head.next)
```

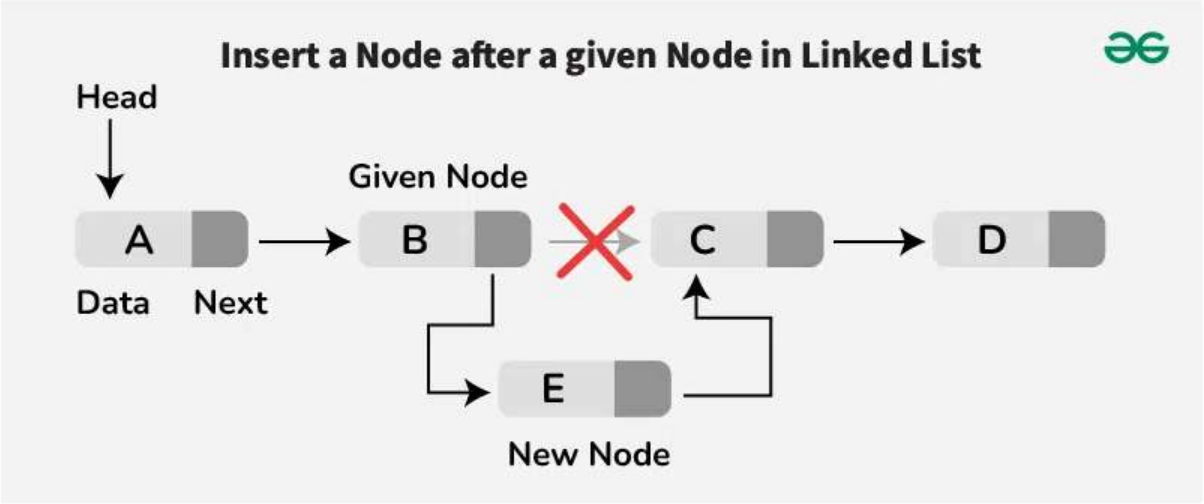
- Search an element:

- Iterative: time complexity O(n), space O(1)
- Recursive: time complexity O(n), space O(n)

- Insertion:
  - At the front: make teh first node linked to teh new one. Remove the head from teh original first node. Make the new node as teh head.

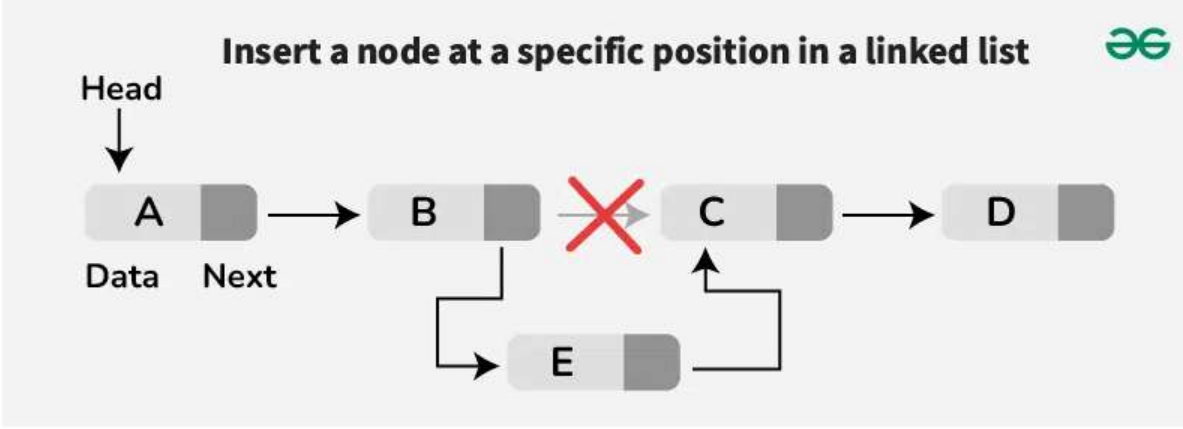


  - After a given node: initialize curr to traverse the list. Loop to find the node with data equal to key. Create a new node with the new data. Make the next pointer of the new node as next of the given one. Update the next pointer of the given node to the new one.



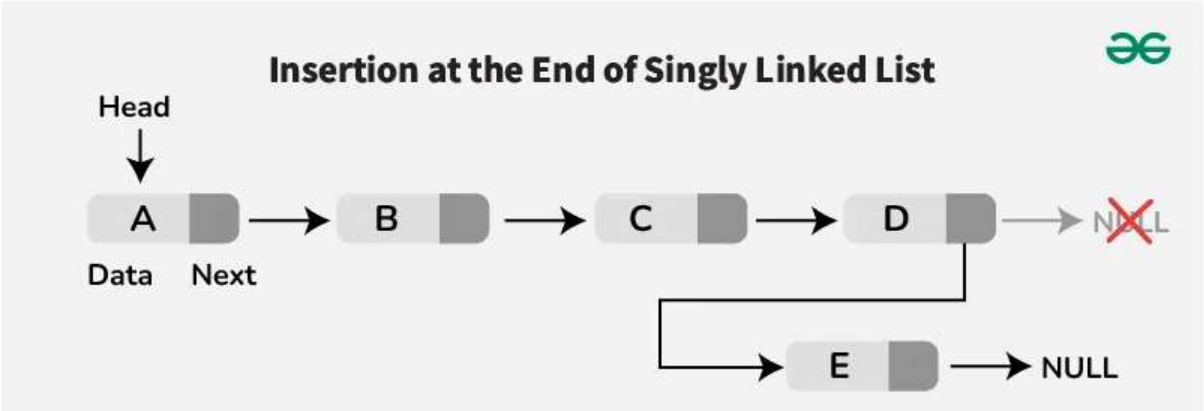
Insertion after a given node in Linked List

  - Before a given node: point the next pointer of the new node to the given one, point the next pointer of the previous node to the new one. If the key is in the head, update the head to point to the new node.
  - At a specific position: once all position -1 nodes are traversed, allocate memory and the given data to the new node. Point the next pointer to the next of the current node and pointe the next pointer of the current node to the new one.



Insertion at specific position in Linked List

  - At the end:



- Deletion:
  - At the beginning: set the head to the second node
  - At a specific position: set the next pointer of the n-1 to point the n+1
  - At the end: set the second-last node's next pointer to NULL.
- Write a function to get Nth node from the end:
  - Length-based traversal: O(n) time and O(1) space. The idea is to count the element of the list and then return the (len - k + 1) nodes from the beginning.
  - Two pointers: O(n) time and O(1) space. Move ref to the kth node from the start. Now move both main and ref until ref reaches the last node. Now return the data of the main.

```
class Node:
    def __init__(self, val=0):
        self.val = val
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    # Slides 53-54: Insert at Head
    def insert_head(self, val):
        new_node = Node(val)
        new_node.next = self.head
        self.head = new_node

    # Slides 55-62: Insert at Tail (O(N) unless you keep a tail pointer)
    def insert_tail(self, val):
        new_node = Node(val)
        if not self.head:
            self.head = new_node
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node

    # Slides 63-64: Search
    def search(self, key):
        curr = self.head
        while curr:
            if curr.val == key:
                return True
            curr = curr.next
        return False
```

```
    # Slides 69-72: Delete by Key
    def delete_key(self, key):
        curr = self.head
        prev = None
        while curr:
            if curr.val == key:
                if prev:
                    prev.next = curr.next # Bypass node
                else:
                    self.head = curr.next # Delete head
                return # No need to call free() in Python!
            prev = curr
            curr = curr.next

    # Slides 96-105: Reverse List (Very common LeetCode question)
    def reverse(self):
        prev = None
        curr = self.head
        while curr:
            next_temp = curr.next
            curr.next = prev
            prev = curr
            curr = next_temp
        self.head = prev
```